

DATABASE TESTING CONCEPTS

Field validation, UI-to-DB sync protocols, data integrity audits, and ACID property verification.

REGISTRATION
Data Insertion

AUDIT QUERY
Integrity Rules

UI VS DB
State Sync

ACID TEST
Transactions

1. User Registration Flow (Food App / Zomato)

BACKEND VERIFICATION

Verify that user details (username, email, phone) submitted via a registration form are accurately persisted in MySQL.

```
-- Step 1: Target Database & Table Definition
CREATE TABLE users (
  user_id INT AUTO_INCREMENT PRIMARY KEY,
  username VARCHAR(50) NOT NULL,
  email VARCHAR(100) UNIQUE NOT NULL,
  phone VARCHAR(15) NOT NULL,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Step 2: Query to verify record creation following form submission
SELECT * FROM users WHERE email = 'aarav@example.com';
```

Verification Result Set:

user_id	username	email	phone	created_at
1	Aarav Sharma	aarav@example.com	9876543210	2026-09-23 10:15:00

2. Data Integrity Audit (Flipkart-Style Orders)

SQL AUDIT

Audit orders table to identify invalid records where status = 'delivered' but quantity <= 0 or item_name IS NULL.

```
-- Query to fetch rule-violating records
SELECT order_id, user_id, item_name, quantity, status
FROM orders
WHERE status = 'delivered'
  AND (quantity <= 0 OR quantity IS NULL OR item_name IS NULL OR TRIM(item_name) = '');
```

Summary Health Check Query:

```
SELECT
  COUNT(*) AS total_delivered,
  SUM(CASE WHEN quantity > 0 AND item_name IS NOT NULL THEN 1 ELSE 0 END) AS valid_order,
  SUM(CASE WHEN quantity <= 0 OR item_name IS NULL THEN 1 ELSE 0 END) AS invalid_order
FROM orders WHERE status = 'delivered';
```

3. UI vs. Database Sync Testing Protocol

FRONTEND / DB SYNC

Test whether manually modified database records immediately reflect in the user interface upon screen refresh.

Step	Action	Query / Execution	Observed / Expected Outcome
1. UI Fetch	Open Profile Screen	App calls GET /api/v1/profile	UI displays: Priya Patel (priya@test.com)
2. DB Inspect	Query Backend Row	SELECT name FROM users WHERE user_id = 45;	Database returns: Priya Patel
3. DB Modify	Manual SQL Update	UPDATE users SET name = 'Priya Shah' WHERE user_id = 45;	Row updated directly in MySQL
4. Refresh UI	Pull-to-refresh App	Re-trigger user fetch API	PASS: UI displays Priya Shah FAIL: UI retains stale name (Cache Issue)

4. Spotify-Style Playlist Field-Level Validation

DATA QUALITY AUDIT

Audit the playlist table to detect empty names or invalid/future created_at timestamps.

```
SELECT playlist_id, playlist_name, created_by, created_at,
CASE
    WHEN playlist_name IS NULL OR TRIM(playlist_name) = '' THEN 'Empty Name'
    WHEN created_at IS NULL OR created_at > CURRENT_TIMESTAMP THEN 'Invalid Date'
    ELSE 'Valid'
END AS audit_status
FROM playlist
WHERE playlist_name IS NULL OR TRIM(playlist_name) = ''
    OR created_at IS NULL OR created_at > CURRENT_TIMESTAMP;
```

5. ACID Properties (BookMyShow Ticket Booking Example)

TRANSACTIONAL INTEGRITY

Real-world scenario: Booking Seat **A12** for ₹300 on BookMyShow.

Atomicity

Either both deducting ₹300 and reserving seat A12 succeed, or if payment fails, the transaction completely rolls back and A12 remains unbooked.

Consistency

Total available seats decrease by exactly 1, preserving database rules without leaving the system in an invalid state.

Isolation

Concurrent booking attempts on seat A12 at the exact same millisecond are processed sequentially so only one user gets the seat.

Durability

Once the booking confirmation screen appears, the ticket record persists in storage and will survive immediate server crashes.